Iker Garcia Morales and Roberto Miguel Martinez Munoz
November 5th, 2026
EE-4322-01
Dr. Gilmour

Final Project Overview: AI Accelerator for Handwritten Digit Recognition

Project Description

The goal of this project is to design and implement a small artificial intelligence (AI) accelerator on the DE2-115 FPGA board that recognizes handwritten digits from 0 to 9. The user draws a digit on an enlarged 8 × 8 grid on a VGA monitor using a PS/2 mouse, presses a Classify button, and the FPGA computes a prediction with a neural network and shows the result on the seven-segment (HEX) display. The network is a two-layer fully connected model (64 → 16 → 10) trained beforehand in Python; its weights are quantized to 8-bit signed integers and stored on the FPGA as .mif files, so during the demonstration all inference runs on the FPGA with no dependency on a PC. The hardware will be written mostly in SystemVerilog, with Quartus Prime megafunctions for the on-chip memories. The project is designed as a real accelerator: a single multiply–accumulate (MAC) baseline will be built first and then scaled to a parallel array of up to 16 MACs so that latency and FPGA resource usage can be measured and compared.

System Overview

Figure 1 shows the top-level block diagram. Offline, a Python script trains the network on a digits dataset binarized the same way as the user-drawn grid, quantizes the weights and biases, and generates the .mif files loaded into the weight ROM when the board is programmed. Live, the input controller decodes the PS/2 mouse packets, tracks the cursor, and writes the drawn cells into the pixel RAM, while the VGA controller generates 640 × 480 timing and renders the grid, cursor, and result. When the user presses Classify (push buttons KEY0–KEY3 handle Classify, Clear, and reset), the accelerator reads the pixel RAM and weight ROM, computes the two layers, and sends the predicted digit to the HEX display.

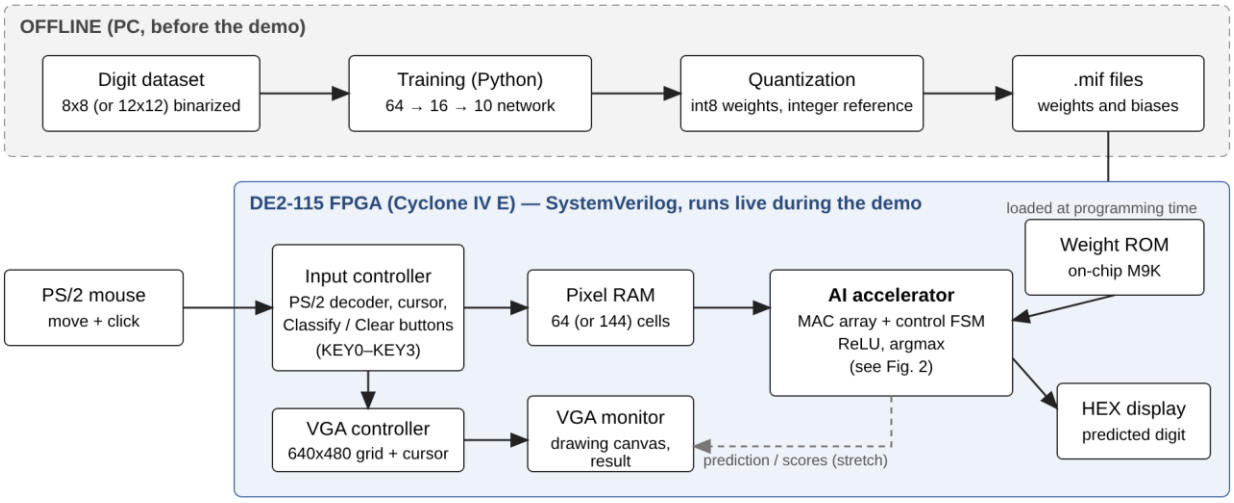


Fig. 1: Top Level Block Diagram of the Digit Recognition System

AI Accelerator Architecture

Figure 2 shows the accelerator in more detail. Inputs and weights are signed 8-bit integers, and each processing element (PE) performs an 8 × 8 signed multiplication with a 32-bit accumulator so that no overflow is possible. The operand-feed block generates the RAM and ROM addresses and registers the operands for the MAC array. After each neuron sum, the bias is added, the result is rescaled and saturated back to 8 bits, and ReLU is applied. The 16 outputs of layer 1 are fed back as the inputs of layer 2, and the 10 outputs of layer 2 go to the argmax block, which selects the most likely digit. A control FSM sequences the process with a start/busy/done handshake, holding the image memory stable while busy.

The RTL is parameterized for 1 PE (baseline) or 16 PEs (logical 4 × 4 array), and the arithmetic will be verified bit-for-bit against an integer-only Python reference model.

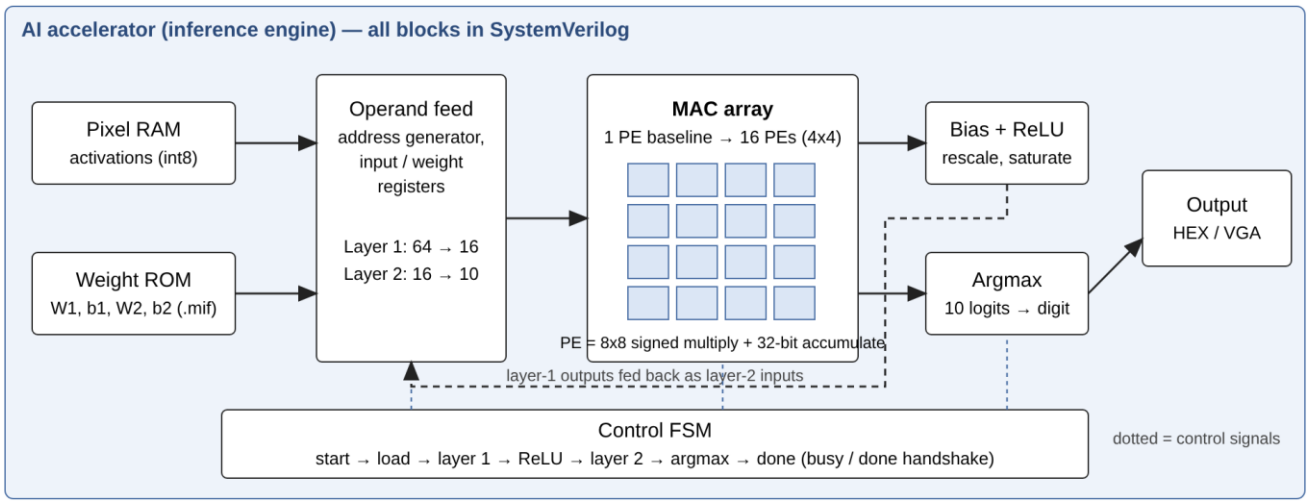


Fig. 2: AI Accelerator Block Diagram

Features

Minimum deliverable (must work): PS/2 mouse drawing on an 8 × 8 VGA grid (draw, erase, clear); pre-trained 64 → 16 → 10 model in on-chip ROM; 1-MAC inference on the DE2-115 with the prediction on the HEX display; exact agreement with the quantized Python reference on at least 50 test vectors.

Planned features (we expect to finish): scaling the accelerator to a 16-MAC parallel array with the same weights; measuring cycles and logic/DSP/memory usage for 1 vs. 16 MACs; showing the prediction and all ten scores on the VGA screen (VGA-only output, without the HEX display); and evaluating a 12 × 12 input grid (144 → 16 → 10).

Stretch goals (if time allows): first, user-driven training, where the user draws a digit and types its label so the model can be retrained with the user's own handwriting (samples collected on the board, retrained in Python, ROM reloaded). Second, and most ambitious, extending the classifier to recognize letters (A–Z) and, one character at a time, short words such as a name. This needs a larger output layer and training set, so it comes after the digit recognizer works.

Role Distribution

Both team members want to be involved in the machine-learning part in order to learn it, so training and quantization are shared. Roberto will lead the accelerator architecture, with Iker taking part of the accelerator design as well to gain experience on that side. Quartus Prime implementation and verification will be split flexibly depending on progress. Interface definitions, board testing, reports, presentation, and video are shared (Table 1).

Table 1: Planned Role Distribution

Area	Roberto Miguel Martinez Munoz	Iker Garcia Morales
Machine learning (shared)	Network definition and training script, int8 quantization, bit-exact integer reference model, .mif generation.	Dataset binarization matching the VGA grid, 8 × 8 vs. 12 × 12 accuracy comparison, collecting user drawings for retraining.
Accelerator architecture	Lead: MAC/PE design, 16-MAC array and dataflow, bias/ReLU/saturation, argmax, cycle and resource measurements.	Control FSM and layer sequencing, address generation and memory mapping (pixel RAM, weight ROM), handshake.
User interface	Classify/Clear buttons and HEX output; VGA-only result display.	VGA timing and grid rendering, PS/2 mouse decoder, cursor, and drawing canvas.
Quartus and verification (flexible)	SystemVerilog testbenches for the arithmetic and equivalence with the Python reference.	Top-level integration, pin assignment, timing closure, and testing on the DE2-115.